

Inteligencia Artificial

Laboratorio 4 - Greedy

1 Enunciado

En base a su código del Laboratorio 2 deberá replicar la función BFS y modificarla para implementar el algoritmo BeFSGreedy. El programa resultante debe recibir como parámetros un archivo **txt** correspondiente a un mapa y un par de coordenadas correspondientes al punto de inicio y termino de un camino. El programa debe utilizar el algoritmo **BeFSGreedy** para encontrar un camino, considerando distancia *Manhattan*, entre dichas coordenadas e imprimir por consola.

Este laboratorio se evaluara en forma de **checklist** a través de la plataforma **GitHub**, Utilice **Commit** individuales, deben tener un nombre y/o descripción apropiado.

1. Use commits en GitHub para registrar sus cambios
2. Cree la función BeFSGreedy
3. Implemente una función de heurística.
4. Implemente un sistema que permita guardar las posiciones en una estructura y extraerlas en base al valor de su heurística. (*investigar Priority Queue como estructura de guardado y Functor para comparacion*)
5. Complete la función **BeFSGreedy**.
6. Imprima por consola el mapa mostrando el camino encontrado por el algoritmo BeFSGreedy.

1.1 Código

Código necesario para *Search.h*

```
static std::vector<std::pair<int,int>> Greedy(const Map& map, std::pair<
    int,int> start, std::pair<int,int> goal); // finds path using Greedy
    Best-First Search
static float Heuristic(std::pair<int,int> start, std::pair<int,int> goal);
    // Calculates heuristic value between two points
```

Código necesario para implementar *Greedy* en *Search.cpp*

```
std::vector<std::pair<int,int>> Search::greedyBFS(const Map& map,
    std::pair<int,int> start, std::pair<int,int> goal) {
    // IMPLEMENTAR
    return {};
}
```

Código necesario para implementar *Heuristica* en *Search.cpp*

```
float Search::Heuristic(std::pair<int,int> start, std::pair<int,int> goal)
{
    // IMPLEMENTAR
    // Debe calcular y devolver un valor heuristico
    return 0.0f;
}
```

1.2 Ejemplo mapa

```
5 5
11111
10001
10001
10001
11111
```

1.3 Ejemplo input

```
./program map.txt 1 1 3 3
```

1.4 Ejemplo output

```
Dist: 4
11111
12001
14001
14431
11111
```

2 Opcional

- Actualmente los mapas contienen valores 0 para los espacios que se pueden recorrer y 1 para muros o camino intransitable, pero que sucederá con los algoritmos si utilizamos mapas con otro formato, ¿Se puede utilizar directamente los algoritmos en *mapas de alturas*?
- ¿Que cambiaría en la implementación si por ejemplo consideráramos que una diferencia de alturas mayor o igual a 4 es un risco (intransitable)?

Ejemplo mapa de alturas

```
6 6
7 9 3 3 8 4
2 0 5 5 1 2
0 6 3 6 1 8
2 0 3 4 4 1
5 8 6 4 8 8
2 6 4 9 4 1
```

Cada casilla guarda el valor de la altura a la cual se encuentra. El coste de moverse de una casilla a otra se ve afectado por la diferencia de altura entre las casillas. Moverse en subida aumenta el coste, mientras que moverse en bajada lo reduce según la formula $Cost * (1 - diff/10)$.